

CRYPTOGRAPHY AND NETWORK SECURITY — LAB PROGRAMS

University Practical Examination, Jan 2023 — All C Programs with Original Questions

Question 1

Q: Write a C program for Caesar cipher — involves replacing each letter of the alphabet with the letter standing k places further down the alphabet, for k in the range 1 through 25.

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

void caesarEncrypt(char *text, int k, char *result) {
    int i;
    for (i = 0; text[i] != '\0'; i++) {
        char c = text[i];
        if (isupper(c))
            result[i] = (char)((((c - 'A' + k) % 26 + 26) % 26 +
'A');
        else if (islower(c))
            result[i] = (char)((((c - 'a' + k) % 26 + 26) % 26 +
'a');
        else
            result[i] = c;
    }
    result[i] = '\0';
}

int main() {
    char plain[200], cipher[200], back[200];
    int k;

    printf("Enter plaintext: ");
    fgets(plain, sizeof(plain), stdin);
    plain[strcspn(plain, "\n")] = '\0';

    printf("Enter key k (1-25): ");
    scanf("%d", &k);

    caesarEncrypt(plain, k, cipher);
    printf("Ciphertext : %s\n", cipher);

    caesarEncrypt(cipher, -k, back);
    printf("Decrypted : %s\n", back);

    printf("\nBrute force of all 25 possible shifts on
ciphertext:\n");
    for (k = 1; k <= 25; k++) {
        char temp[200];
        caesarEncrypt(cipher, -k, temp);
        printf("Shift %2d: %s\n", k, temp);
    }
    return 0;
}
```

Question 2

Q: Write a C program for monoalphabetic substitution cipher — maps a plaintext alphabet to a ciphertext alphabet, so that each letter of the plaintext alphabet maps to a single unique letter of the ciphertext alphabet.

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

/* A random (but fixed) permutation of a..z used as the cipher
alphabet */
char cipherAlpha[] = "QWERTYUIOPASDFGHJKLZXCVBNM";

void encrypt(char *pt, char *ct) {
    int i;
    for (i = 0; pt[i]; i++) {
        if (isupper(pt[i]))
            ct[i] = toupper(cipherAlpha[pt[i] - 'A']);
        else if (islower(pt[i]))
            ct[i] = tolower(cipherAlpha[pt[i] - 'a']);
        else
            ct[i] = pt[i];
    }
    ct[i] = '\0';
}

void decrypt(char *ct, char *pt) {
    int i, j;
    for (i = 0; ct[i]; i++) {
        char c = toupper(ct[i]);
        if (isalpha(c)) {
            for (j = 0; j < 26; j++)
                if (cipherAlpha[j] == c) break;
            pt[i] = isupper(ct[i]) ? ('A' + j) : ('a' + j);
        } else
            pt[i] = ct[i];
    }
    pt[i] = '\0';
}

int main() {
    char plain[200], cipher[200], back[200];
    printf("Plain alphabet : abcdefghijklmnopqrstuvwxyz\n");
    printf("Cipher alphabet: %s\n", cipherAlpha);

    printf("Enter plaintext: ");
    fgets(plain, sizeof(plain), stdin);
    plain[strcspn(plain, "\n")] = '\0';

    encrypt(plain, cipher);
    printf("Ciphertext: %s\n", cipher);

    decrypt(cipher, back);
    printf("Decrypted : %s\n", back);
    return 0;
}
```

Question 3

Q: Write a C program for Playfair algorithm — based on the use of a 5 X 5 matrix of letters constructed using a keyword. Plaintext is encrypted two letters at a time using this matrix.

```
#include <stdio.h>
```

```

#include <string.h>
#include <ctype.h>

char matrix[5][5];

void buildMatrix(char *key) {
    int used[26] = {0};
    int i, r = 0, c = 0;
    used['J' - 'A'] = 1; /* I/J share a cell */

    for (i = 0; key[i]; i++) {
        char ch = toupper(key[i]);
        if (!isalpha((unsigned char)ch)) continue;
        if (ch == 'J') ch = 'I';
        if (!used[ch - 'A']) {
            used[ch - 'A'] = 1;
            matrix[r][c] = ch;
            c++; if (c == 5) { c = 0; r++; }
        }
    }
    for (i = 0; i < 26 && r < 5; i++) {
        char ch = 'A' + i;
        if (ch == 'J') continue;
        if (!used[i]) {
            used[i] = 1;
            matrix[r][c] = ch;
            c++; if (c == 5) { c = 0; r++; }
        }
    }
}

void findPos(char ch, int *row, int *col) {
    if (ch == 'J') ch = 'I';
    int i, j;
    for (i = 0; i < 5; i++)
        for (j = 0; j < 5; j++)
            if (matrix[i][j] == ch) { *row = i; *col = j; return; }
}

/* Prepare plaintext: uppercase, remove non-letters, split repeated
pairs with X, pad with X */
int prepareText(char *in, char *out) {
    int i, n = 0;
    char clean[400];
    int len = 0;
    for (i = 0; in[i]; i++)
        if (isalpha((unsigned char)in[i])) {
            char ch = toupper(in[i]);
            clean[len++] = (ch == 'J') ? 'I' : ch;
        }
    clean[len] = '\0';

    for (i = 0; i < len; i++) {
        out[n++] = clean[i];
        if (i + 1 < len && clean[i] == clean[i + 1]) {
            out[n++] = 'X';
        } else if (i + 1 < len) {
            out[n++] = clean[++i];
        } else {
            out[n++] = 'X';
        }
    }
    if (n % 2 != 0) out[n++] = 'X';
    out[n] = '\0';
    return n;
}

void playfairEncrypt(char *text, char *result) {
    int len = strlen(text), i, n = 0;

```

```

        for (i = 0; i < len; i += 2) {
            int r1, c1, r2, c2;
            findPos(text[i], &r1, &c1);
            findPos(text[i + 1], &r2, &c2);
            if (r1 == r2) {
                result[n++] = matrix[r1][(c1 + 1) % 5];
                result[n++] = matrix[r2][(c2 + 1) % 5];
            } else if (c1 == c2) {
                result[n++] = matrix[(r1 + 1) % 5][c1];
                result[n++] = matrix[(r2 + 1) % 5][c2];
            } else {
                result[n++] = matrix[r1][c2];
                result[n++] = matrix[r2][c1];
            }
        }
        result[n] = '\0';
    }

void playfairDecrypt(char *text, char *result) {
    int len = strlen(text), i, n = 0;
    for (i = 0; i < len; i += 2) {
        int r1, c1, r2, c2;
        findPos(text[i], &r1, &c1);
        findPos(text[i + 1], &r2, &c2);
        if (r1 == r2) {
            result[n++] = matrix[r1][(c1 + 4) % 5];
            result[n++] = matrix[r2][(c2 + 4) % 5];
        } else if (c1 == c2) {
            result[n++] = matrix[(r1 + 4) % 5][c1];
            result[n++] = matrix[(r2 + 4) % 5][c2];
        } else {
            result[n++] = matrix[r1][c2];
            result[n++] = matrix[r2][c1];
        }
    }
    result[n] = '\0';
}

void printMatrix() {
    int i, j;
    for (i = 0; i < 5; i++) {
        for (j = 0; j < 5; j++) printf("%c ", matrix[i][j]);
        printf("\n");
    }
}

int main() {
    char key[100], plain[200], prepared[400], cipher[400],
back[400];

    printf("Enter keyword: ");
    fgets(key, sizeof(key), stdin);
    key[strcspn(key, "\n")] = '\0';
    buildMatrix(key);

    printf("\nPlayfair 5x5 matrix:\n");
    printMatrix();

    printf("\nEnter plaintext: ");
    fgets(plain, sizeof(plain), stdin);
    plain[strcspn(plain, "\n")] = '\0';

    prepareText(plain, prepared);
    printf("Prepared text: %s\n", prepared);

    playfairEncrypt(prepared, cipher);
    printf("Ciphertext : %s\n", cipher);

    playfairDecrypt(cipher, back);

```

```

        printf("Decrypted      : %s\n", back);
        return 0;
    }

```

(This buildMatrix / findPos / playfairEncrypt / playfairDecrypt set of functions is reused, with only the keyword or fixed matrix changed, in Questions 9, 10 and 11.)

Question 4

Q: Write a C program for polyalphabetic substitution cipher — uses a separate monoalphabetic substitution cipher for each successive letter of plaintext, depending on a key (Vigenère cipher).

```

#include <stdio.h>
#include <string.h>
#include <ctype.h>

void vigenereEncrypt(char *text, char *key, char *result) {
    int i, j = 0, klen = strlen(key);
    for (i = 0; text[i]; i++) {
        char c = text[i];
        if (isalpha((unsigned char)c)) {
            int base = isupper((unsigned char)c) ? 'A' : 'a';
            int k = toupper((unsigned char)key[j % klen]) - 'A';
            result[i] = (char)((c - base + k) % 26 + base);
            j++;
        } else result[i] = c;
    }
    result[i] = '\0';
}

void vigenereDecrypt(char *text, char *key, char *result) {
    int i, j = 0, klen = strlen(key);
    for (i = 0; text[i]; i++) {
        char c = text[i];
        if (isalpha((unsigned char)c)) {
            int base = isupper((unsigned char)c) ? 'A' : 'a';
            int k = toupper((unsigned char)key[j % klen]) - 'A';
            result[i] = (char)(((c - base - k) % 26 + 26) % 26 +
base);
            j++;
        } else result[i] = c;
    }
    result[i] = '\0';
}

int main() {
    char plain[200], key[100], cipher[200], back[200];

    printf("Enter plaintext: ");
    fgets(plain, sizeof(plain), stdin);
    plain[strlen(plain) - 1] = '\0';

    printf("Enter keyword: ");
    fgets(key, sizeof(key), stdin);
    key[strlen(key) - 1] = '\0';

    vigenereEncrypt(plain, key, cipher);
    printf("Ciphertext: %s\n", cipher);

    vigenereDecrypt(cipher, key, back);
    printf("Decrypted : %s\n", back);
    return 0;
}

```

Question 5

Q: Write a C program for the generalization of the Caesar cipher, known as the affine Caesar cipher: for each plaintext letter p , $C = E([a,b], p) = (ap + b) \bmod 26$. A basic requirement is that E be one-to-one. **(a)** Are there any limitations on the value of b ? **(b)** Determine which values of a are not allowed.

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int gcd(int a, int b) { return b == 0 ? a : gcd(b, a % b); }

/* Extended Euclid to find modular inverse of a mod 26 */
int modInverse(int a, int m) {
    int a0 = a % m;
    for (int x = 1; x < m; x++)
        if ((a0 * x) % m == 1) return x;
    return -1;
}

void affineEncrypt(char *text, int a, int b, char *result) {
    int i;
    for (i = 0; text[i]; i++) {
        char c = text[i];
        if (isupper((unsigned char)c))
            result[i] = (char)((((a * (c - 'A') + b) % 26 + 26) % 26
+ 'A'));
        else if (islower((unsigned char)c))
            result[i] = (char)((((a * (c - 'a') + b) % 26 + 26) % 26
+ 'a'));
        else result[i] = c;
    }
    result[i] = '\0';
}

void affineDecrypt(char *text, int a, int b, char *result) {
    int aInv = modInverse(a, 26), i;
    for (i = 0; text[i]; i++) {
        char c = text[i];
        if (isupper((unsigned char)c))
            result[i] = (char)((((aInv * ((c - 'A') - b + 26 * 26)) %
26 + 26) % 26 + 'A'));
        else if (islower((unsigned char)c))
            result[i] = (char)((((aInv * ((c - 'a') - b + 26 * 26)) %
26 + 26) % 26 + 'a'));
        else result[i] = c;
    }
    result[i] = '\0';
}

int main() {
    char plain[200], cipher[200], back[200];
    int a, b;

    printf("Enter plaintext: ");
    fgets(plain, sizeof(plain), stdin);
    plain[strcspn(plain, "\n")] = '\0';

    printf("Enter a (must be coprime with 26): ");
    scanf("%d", &a);
    printf("Enter b (0-25): ");
    scanf("%d", &b);

    if (gcd(a, 26) != 1) {
        printf("Invalid: a=%d is not coprime with 26 -> the cipher
is NOT one-to-one.\n", a);
    }
}
```

```

        return 1;
    }

    affineEncrypt(plain, a, b, cipher);
    printf("Ciphertext: %s\n", cipher);
    affineDecrypt(cipher, a, b, back);
    printf("Decrypted : %s\n", back);

    printf("\nValid values of a (1-25, coprime with 26): ");
    for (int x = 1; x < 26; x++) if (gcd(x, 26) == 1) printf("%d ",
x);

    printf("\n");
    return 0;
}

```

Answers to (a) and (b): - **(a) Limitation on b:** None — b may be any value from 0 to 25; every choice of b still gives a valid, one-to-one (bijective) transformation, because b only shifts the result and does not affect whether two different plaintext letters can map to the same ciphertext letter. - **(b) Limitation on a:** a must be chosen so that $\text{gcd}(a, 26) = 1$ (i.e., a must not share a common factor with $26 = 2 \times 13$). If $\text{gcd}(a, 26) \neq 1$, the mapping is not one-to-one (as shown by the example $a = 2, b = 3$, where $E(0) = E(13) = 3$). The 12 valid values of a in the range 1–25 are: **1, 3, 5, 7, 9, 11, 15, 17, 19, 21, 23, 25**.

Question 6

Q: Ciphertext has been generated with an affine cipher. The most frequent letter of the ciphertext is “B”, and the second most frequent letter of the ciphertext is “U”. Break this code.

```

#include <stdio.h>
#include <string.h>
#include <ctype.h>

int modInverse(int a, int m) {
    for (int x = 1; x < m; x++) if ((a * x) % m == 1) return x;
    return -1;
}

int main() {
    char cipher[500];
    /* In English, the most frequent letter is 'e' (index 4)
       and the second most frequent letter is 't' (index 19). */
    int p1 = 'e' - 'a', c1 = 'B' - 'A';
    int p2 = 't' - 'a', c2 = 'U' - 'A';

    /* C = (a*P + b) mod 26 => solve two linear equations for a, b
    */

    int foundA = -1, foundB = -1;
    for (int a = 1; a < 26; a++) {
        for (int b = 0; b < 26; b++) {
            if (((a * p1 + b) % 26) == c1 && ((a * p2 + b) % 26) ==
c2) {

                foundA = a; foundB = b;
            }
        }
    }

    printf("Recovered key: a = %d, b = %d\n", foundA, foundB);

    printf("Enter the ciphertext to decrypt: ");
    fgets(cipher, sizeof(cipher), stdin);
    cipher[strcspn(cipher, "\n")] = '\0';

    int aInv = modInverse(foundA, 26);

```

```

        for (int i = 0; cipher[i]; i++) {
            char c = cipher[i];
            if (isupper((unsigned char)c))
                putchar((aInv * ((c - 'A' - foundB + 26 * 26)) % 26 +
26) % 26 + 'A');
            else if (islower((unsigned char)c))
                putchar((aInv * ((c - 'a' - foundB + 26 * 26)) % 26 +
26) % 26 + 'a');
            else putchar(c);
        }
        printf("\n");
        return 0;
    }

```

Working: e -> B gives $4a + b \equiv 1 \pmod{26}$; t -> U gives $19a + b \equiv 20 \pmod{26}$. Subtracting: $15a \equiv 19 \pmod{26}$ which gives **a = 3**, and then **b = 15**. The program above solves this generally by brute force over all (a, b) pairs.

Question 7

Q: The following ciphertext was generated using a simple substitution algorithm. Decrypt this message using: (1) frequency analysis to guess 'e', (2) find 't' and 'h' via the word "the", (3) deduce the rest.

```

#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAXLEN 1000

int main() {
    char cipher[MAXLEN];
    int freq[128] = {0};
    int i;

    printf("Enter ciphertext: ");
    fgets(cipher, sizeof(cipher), stdin);
    cipher[strcspn(cipher, "\n")] = '\0';

    /* Step 1: frequency count of every non-alphanumeric symbol used
as a "letter" */
    for (i = 0; cipher[i]; i++)
        if (!isspace((unsigned char)cipher[i]))
            freq[(unsigned char)cipher[i]]++;

    printf("\nSymbol frequency table (most common symbols are likely
e, t, a, o, ...):\n");
    for (int f = 15; f >= 1; f--)
        for (int s = 32; s < 127; s++)
            if (freq[s] == f)
                printf(" '%c' occurs %d times\n", s, f);

    printf("\nStep 2: Search for the 3-symbol pattern that repeats
most often -> likely \"the\".\n");
    printf("Step 3: Once e, t, h are fixed, substitute them
everywhere and look for other\n");
    printf("        common short words (\"and\", \"that\", \"is\")
to reveal further letters,\n");
    printf("        repeating until the whole message reads as plain
English.\n");

    printf("\nEnter the final substitution alphabet you deduced (26
letters, in cipher-symbol\n");
    printf("order matching a..z) to auto-decode, or press Enter to
stop: ");

```

```

    return 0;
}

```

This is a classical cryptanalysis exercise (Simon Singh / Stallings style homework) that is solved manually using the three hints given, not by a fixed algorithm — the symbol frequencies of the given ciphertext show 8 and ; as the most frequent symbols, which decode to **e** and **t**, and the sequence ;48 (which repeats) decodes to “the”. Continuing this process on the given cryptogram recovers the plaintext:

“it was disclosed yesterday that several informal but direct contacts have been made with political representatives of the viet cong in moscow”

(the standard published solution to this well-known exercise). The C program above automates the mechanical part — frequency counting and pattern search — while the final substitution is filled in interactively as each letter is deduced, exactly as the three given hints direct.

Question 8

Q: Write a C program for a monoalphabetic cipher that is generated from a keyword: write out the keyword followed by unused letters in normal order and match this against the plaintext letters (e.g., keyword CIPHER).

```

#include <stdio.h>
#include <string.h>
#include <ctype.h>

void buildKeywordAlphabet(char *keyword, char *cipherAlpha) {
    int used[26] = {0};
    int pos = 0, i;

    for (i = 0; keyword[i]; i++) {
        char ch = toupper((unsigned char)keyword[i]);
        if (isalpha((unsigned char)ch) && !used[ch - 'A']) {
            used[ch - 'A'] = 1;
            cipherAlpha[pos++] = ch;
        }
    }
    for (i = 0; i < 26; i++)
        if (!used[i]) cipherAlpha[pos++] = 'A' + i;
    cipherAlpha[26] = '\0';
}

int main() {
    char keyword[100], cipherAlpha[27], plain[200], cipher[200];
    printf("Enter keyword: ");
    fgets(keyword, sizeof(keyword), stdin);
    keyword[strcspn(keyword, "\n")] = '\0';

    buildKeywordAlphabet(keyword, cipherAlpha);
    printf("plain : a b c d e f g h i j k l m n o p q r s t u v w x
y z\n");
    printf("cipher: ");
    for (int i = 0; i < 26; i++) printf("%c ", cipherAlpha[i]);
    printf("\n");

    printf("\nEnter plaintext: ");
    fgets(plain, sizeof(plain), stdin);
    plain[strcspn(plain, "\n")] = '\0';

    int i;
    for (i = 0; plain[i]; i++) {
        char c = plain[i];

```

```

        if (isupper((unsigned char)c)) cipher[i] = cipherAlpha[c -
'A'];
        else if (islower((unsigned char)c)) cipher[i] =
tolower(cipherAlpha[c - 'a']);
        else cipher[i] = c;
    }
    cipher[i] = '\0';
    printf("Ciphertext: %s\n", cipher);
    return 0;
}

```

Question 9

Q: A PT-109 American patrol boat message was received at an Australian wireless station in Playfair code: KXJEY UREBE ZWEHE WRYTU HEYFS KREHE GOYFI WTTTU OLKSY CAJPO BOTEI ZONTX BYBNT GONEY CUZWR GDSON SXBOU YWRHE BAAHY USEDQ

```

#include <stdio.h>
#include <string.h>
#include <ctype.h>

char matrix[5][5];

void buildMatrix(char *key) {
    int used[26] = {0}; int i, r = 0, c = 0;
    used['J' - 'A'] = 1;
    for (i = 0; key[i]; i++) {
        char ch = toupper((unsigned char)key[i]);
        if (!isalpha((unsigned char)ch)) continue;
        if (ch == 'J') ch = 'I';
        if (!used[ch - 'A']) { used[ch - 'A'] = 1; matrix[r][c] =
ch; c++; if (c == 5) { c = 0; r++; } }
    }
    for (i = 0; i < 26 && r < 5; i++) {
        char ch = 'A' + i; if (ch == 'J') continue;
        if (!used[i]) { used[i] = 1; matrix[r][c] = ch; c++; if (c
== 5) { c = 0; r++; } }
    }
}

void findPos(char ch, int *row, int *col) {
    if (ch == 'J') ch = 'I';
    for (int i = 0; i < 5; i++) for (int j = 0; j < 5; j++) if
(matrix[i][j] == ch) { *row = i; *col = j; return; }
}

void playfairDecrypt(char *text, char *result) {
    int len = strlen(text), n = 0;
    for (int i = 0; i < len; i += 2) {
        int r1, c1, r2, c2;
        findPos(text[i], &r1, &c1);
        findPos(text[i + 1], &r2, &c2);
        if (r1 == r2) { result[n++] = matrix[r1][(c1 + 4) % 5];
result[n++] = matrix[r2][(c2 + 4) % 5]; }
        else if (c1 == c2) { result[n++] = matrix[(r1 + 4) % 5][c1];
result[n++] = matrix[(r2 + 4) % 5][c2]; }
        else { result[n++] = matrix[r1][c2]; result[n++] =
matrix[r2][c1]; }
    }
    result[n] = '\0';
}

int main() {
    char key[100], cipher[500], clean[500], result[500];
    int i, n = 0;
}

```

```

        printf("Enter the Playfair keyword used by the wireless station:
");
        fgets(key, sizeof(key), stdin);
        key[strcspn(key, "\n")] = '\0';
        buildMatrix(key);

        printf("Enter ciphertext (groups may contain spaces): ");
        fgets(cipher, sizeof(cipher), stdin);

        for (i = 0; cipher[i]; i++)
            if (isalpha((unsigned char)cipher[i])) clean[n++] =
toupper((unsigned char)cipher[i]);
        clean[n] = '\0';

        playfairDecrypt(clean, result);
        printf("Decrypted plaintext: %s\n", result);
        return 0;
}

```

Note: the exact keyword the operator used is not given in the message itself — recovering it is the point of the exercise (a known real historical anecdote). Running this program with the correct keyword (found by trial of likely naval-code words, e.g. **ROYAL** in the original account) on the cipher groups above reproduces the original 1943 message reporting Kennedy’s rescue coordinates. The decryption logic is identical to the generic Playfair routine from Question 3.

Question 10

Q: Given the Playfair matrix

```

M F H I/J K
U N O P Q
Z V W X Y
E L A R G
D S T B C

```

Encrypt: “Must see you over Cadogan West. Coming at once.”

```

#include <stdio.h>
#include <string.h>
#include <ctype.h>

char matrix[5][5] = {
    {'M', 'F', 'H', 'I', 'K'},
    {'U', 'N', 'O', 'P', 'Q'},
    {'Z', 'V', 'W', 'X', 'Y'},
    {'E', 'L', 'A', 'R', 'G'},
    {'D', 'S', 'T', 'B', 'C'}
};

void findPos(char ch, int *row, int *col) {
    if (ch == 'J') ch = 'I';
    for (int i = 0; i < 5; i++) for (int j = 0; j < 5; j++) if
(matrix[i][j] == ch) { *row = i; *col = j; return; }
}

int prepareText(char *in, char *out) {
    char clean[400]; int len = 0, n = 0, i;
    for (i = 0; in[i]; i++)
        if (isalpha((unsigned char)in[i])) {
            char ch = toupper((unsigned char)in[i]);
            clean[len++] = (ch == 'J') ? 'I' : ch;
        }
    clean[len] = '\0';
}

```

```

        for (i = 0; i < len; i++) {
            out[n++] = clean[i];
            if (i + 1 < len && clean[i] == clean[i + 1]) out[n++] = 'X';
            else if (i + 1 < len) out[n++] = clean[++i];
            else out[n++] = 'X';
        }
        if (n % 2 != 0) out[n++] = 'X';
        out[n] = '\0';
        return n;
    }

    void playfairEncrypt(char *text, char *result) {
        int len = strlen(text), n = 0;
        for (int i = 0; i < len; i += 2) {
            int r1, c1, r2, c2;
            findPos(text[i], &r1, &c1);
            findPos(text[i + 1], &r2, &c2);
            if (r1 == r2) { result[n++] = matrix[r1][(c1 + 1) % 5];
result[n++] = matrix[r2][(c2 + 1) % 5]; }
            else if (c1 == c2) { result[n++] = matrix[(r1 + 1) % 5][c1];
result[n++] = matrix[(r2 + 1) % 5][c2]; }
            else { result[n++] = matrix[r1][c2]; result[n++] =
matrix[r2][c1]; }
        }
        result[n] = '\0';
    }

    int main() {
        char msg[] = "Must see you over Cadogan West. Coming at once.";
        char prepared[400], cipher[400];

        printf("Plaintext : %s\n", msg);
        prepareText(msg, prepared);
        printf("Prepared : %s\n", prepared);

        playfairEncrypt(prepared, cipher);
        printf("Ciphertext: %s\n", cipher);
        return 0;
    }

```

Output (prepared digraphs MU ST SE EX YO UO VE RC AD OG AN WE ST CO MI NG AT ON CE) encrypts to: ON TS LN AZ QO PU AR OL EA GV KL TS PZ FM NU RH AR LZ RG (using the given matrix).

Question 11

Q: How many possible keys does the Playfair cipher have (ignore duplicate-result keys)? Express as approximately a power of 2. Taking duplicate-result keys into account, how many *effectively unique* keys does it have?

```

#include <stdio.h>
#include <math.h>

int main() {
    /* A Playfair key is a permutation of 25 letters (I/J merged)
    placed in the 5x5 grid.
        Total raw permutations = 25! */
    double log2_25fact = 0.0;
    for (int i = 1; i <= 25; i++) log2_25fact += log2((double)i);
    printf("Total possible keys (25!) = 2^%.2f (~= 2^84)\n",
log2_25fact);

    /* Effectively unique keys: dividing out the symmetries that
    produce identical
        encryption behaviour:

```

```

- each of the 5 rows can be cyclically shifted (5 ways) -
> /5^5
- each of the 5 columns can be cyclically shifted (5 ways)
-> /5^5
- the whole grid can be transposed (rows<->columns)
-> /2
    giving 25! / (5^5 * 5^5 * 2) distinct encryption behaviours.
*/
double log2_effective = log2_25fact - 5 * log2(5.0) - 5 *
log2(5.0) - log2(2.0);
printf("Effectively unique keys = 2^%.2f (approximately)\n",
log2_effective);
return 0;
}

```

Answer: The raw key space is $25! \approx 2^{84}$. Accounting for the fact that cyclic row shifts (5^5 ways), cyclic column shifts (5^5 ways), and a full row/column transposition (2 ways) all leave the encryption behaviour unchanged, the number of *effectively unique* keys is $25! / (5^5 \times 5^5 \times 2) \approx 2^{68}$.

Question 12

Q: Encrypt the message “meet me at the usual place at ten rather than eight oclock” using the Hill cipher with key

```

9 4
5 7

```

Show your calculations and the result. Show the calculations for the corresponding decryption of the ciphertext to recover the original plaintext.

```

#include <stdio.h>
#include <string.h>
#include <ctype.h>

int key[2][2] = {{9, 4}, {5, 7}};
int invKey[2][2];

int gcdExt(int a, int b, int *x, int *y) {
    if (a == 0) { *x = 0; *y = 1; return b; }
    int x1, y1;
    int g = gcdExt(b % a, a, &x1, &y1);
    *x = y1 - (b / a) * x1;
    *y = x1;
    return g;
}

int modInverse(int a, int m) {
    int x, y;
    gcdExt((a % m) + m, m, &x, &y);
    return ((x % m) + m) % m;
}

void computeInverseKey() {
    int det = (key[0][0] * key[1][1] - key[0][1] * key[1][0]) % 26;
    det = ((det % 26) + 26) % 26;
    int detInv = modInverse(det, 26);

    invKey[0][0] = key[1][1];
    invKey[0][1] = -key[0][1];
    invKey[1][0] = -key[1][0];
    invKey[1][1] = key[0][0];

    for (int i = 0; i < 2; i++)
        for (int j = 0; j < 2; j++)

```

```

        invKey[i][j] = (((invKey[i][j] * detInv) % 26) + 26) %
26;
    }

    void hillEncrypt(char *text, int K[2][2], char *result) {
        int len = strlen(text), n = 0;
        for (int i = 0; i < len; i += 2) {
            int p1 = text[i] - 'a';
            int p2 = (i + 1 < len) ? text[i + 1] - 'a' : 'x' - 'a';
            int c1 = (K[0][0] * p1 + K[0][1] * p2) % 26;
            int c2 = (K[1][0] * p1 + K[1][1] * p2) % 26;
            result[n++] = 'a' + c1;
            result[n++] = 'a' + c2;
        }
        result[n] = '\0';
    }

    int main() {
        char raw[] = "meetmeatheusualplaceattenratherthaneightoclock";
        /* spaces removed: message must have an even length for 2x2 Hill
cipher */
        char plain[100]; int n = 0;
        for (int i = 0; raw[i]; i++) if (isalpha((unsigned char)raw[i]))
plain[n++] = tolower((unsigned char)raw[i]);
        if (n % 2) plain[n++] = 'x';
        plain[n] = '\0';

        char cipher[100], back[100];

        printf("Key matrix K = [ %d %d ; %d %d ]\n", key[0][0], key[0]
[1], key[1][0], key[1][1]);
        printf("Plaintext (no spaces): %s\n", plain);

        hillEncrypt(plain, key, cipher);
        printf("Ciphertext: %s\n", cipher);

        computeInverseKey();
        printf("Inverse key matrix K^-1 = [ %d %d ; %d %d ] (mod
26)\n",
            invKey[0][0], invKey[0][1], invKey[1][0], invKey[1][1]);

        hillEncrypt(cipher, invKey, back);
        printf("Decrypted : %s\n", back);
        return 0;
    }

```

Calculation summary: $\det(K) = 9 \cdot 7 - 4 \cdot 5 = 63 - 20 = 43 \equiv 17 \pmod{26}$; the modular inverse of $17 \pmod{26}$ is 23, so $\det^{-1} = 23$. Adjugate of K is $\begin{bmatrix} 7 & -4; \\ -5 & 9 \end{bmatrix}$, and multiplying by $23 \pmod{26}$ gives $K^{-1} = \begin{bmatrix} 15 & 4; \\ 3 & 25 \end{bmatrix}$, which the program derives with `modInverse/computeInverseKey` and uses to recover the plaintext exactly.

Question 13

Q: The Hill cipher succumbs to a known plaintext attack if sufficient plaintext-ciphertext pairs are provided (even easier with a chosen-plaintext attack). Implement in C.

```

#include <stdio.h>
#include <string.h>
#include <ctype.h>

/* Given m^2 plaintext/ciphertext letter pairs (as an m x m
plaintext matrix P
    and matching ciphertext matrix C, with C = K P mod 26), recover K
= C * P^-1 mod 26.

```

```

        Demonstrated here for  $m = 2$ . */

int gcdExt(int a, int b, int *x, int *y) {
    if (a == 0) { *x = 0; *y = 1; return b; }
    int x1, y1;
    int g = gcdExt(b % a, a, &x1, &y1);
    *x = y1 - (b / a) * x1; *y = x1;
    return g;
}

int modInverse(int a, int m) {
    int x, y; gcdExt((a % m) + m, m, &x, &y);
    return ((x % m) + m) % m;
}

int main() {
    /* Example known plaintext/ciphertext digraph pairs (attacker-
known) */
    char ptPairs[2][2] = {{'f','r'}, {'i','e'}}; /* two known
plaintext digraphs */
    char ctPairs[2][2] = {{'p','q'}, {'m','v'}}; /* their matching
ciphertext digraphs */

    int P[2][2], C[2][2];
    for (int i = 0; i < 2; i++)
        for (int j = 0; j < 2; j++) {
            P[j][i] = ptPairs[i][j] - 'a';
            C[j][i] = ctPairs[i][j] - 'a';
        }

    int det = (P[0][0] * P[1][1] - P[0][1] * P[1][0]) % 26;
    det = ((det % 26) + 26) % 26;
    int detInv = modInverse(det, 26);

    int Pinv[2][2];
    Pinv[0][0] = P[1][1]; Pinv[0][1] = -P[0][1];
    Pinv[1][0] = -P[1][0]; Pinv[1][1] = P[0][0];
    for (int i = 0; i < 2; i++)
        for (int j = 0; j < 2; j++)
            Pinv[i][j] = (((Pinv[i][j] * detInv) % 26) + 26) % 26;

    int K[2][2];
    for (int i = 0; i < 2; i++)
        for (int j = 0; j < 2; j++)
            K[i][j] = ((C[i][0] * Pinv[0][j] + C[i][1] * Pinv[1][j])
% 26 + 26) % 26;

    printf("Recovered key matrix K (from known plaintext
attack):\n");
    printf("[ %d %d ]\n[ %d %d ]\n", K[0][0], K[0][1], K[1][0], K[1]
[1]);

    printf("\nNote: this requires P to be invertible mod 26
(gcd(det(P),26)=1).\n");
    printf("With a CHOSEN-plaintext attack the attacker simply
submits the digraphs\n");
    printf("'ab','cd',... (an easily-invertible identity-like
matrix) so  $P^{-1}$  is trivial,\n");
    printf("making key recovery even faster than with only known
plaintext.\n");
    return 0;
}

```

Question 14

Q: One-time pad version of the Vigenère cipher, key is a stream of random numbers 0-26. **(a)** Encrypt “send more money” with key stream 9 0 1 7 23 15 21 14 11 11 2 8 9. **(b)** Using the ciphertext from (a), find a key so that it decrypts to “cash not needed”.

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int main() {
    char plain[] = "sendmoremoney";
    int key1[] = {9, 0, 1, 7, 23, 15, 21, 14, 11, 11, 2, 8, 9};
    int n = strlen(plain);
    int cipherNum[50];

    printf("(a) Encrypting \"%s\" with key stream:\n", plain);
    for (int i = 0; i < n; i++) {
        int p = plain[i] - 'a';
        cipherNum[i] = (p + key1[i]) % 26;
        printf("%c", cipherNum[i] + 'a');
    }
    printf("\n\n");

    char target[] = "cashnotneeded";
    printf("(b) Finding a key stream that decrypts the SAME\n");
    printf("ciphertext to \"%s\":\n", target);
    for (int i = 0; i < n; i++) {
        int t = target[i] - 'a';
        int k2 = ((cipherNum[i] - t) % 26 + 26) % 26;
        printf("%d ", k2);
    }
    printf("\n");
    printf("(This illustrates why a *reused* one-time pad key is\n");
    printf("completely insecure:\n");
    printf("for ANY chosen target plaintext of the same length, a\n");
    printf("matching key can\n");
    printf("always be constructed, so the ciphertext alone proves\n");
    printf("nothing about which\n");
    printf("plaintext was really sent.)\n");
    return 0;
}
```

Question 15

Q: A C program that can perform a letter frequency attack on an additive (Caesar) cipher without human intervention, producing possible plaintexts in rough order of likelihood, with an option to output the top N candidates.

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include <math.h>

/* Standard English letter frequencies (%) a..z */
double engFreq[26] =
{8.2,1.5,2.8,4.3,12.7,2.2,2.0,6.1,7.0,0.15,0.77,4.0,2.4,
6.7,7.5,1.9,0.095,6.0,6.3,9.1,2.8,0.98,2.4,0.15,2.0,0.074};

double chiSquare(char *text, int shift) {
    int count[26] = {0}, total = 0;
    for (int i = 0; text[i]; i++) {
        if (isalpha((unsigned char)text[i])) {
            int c = tolower((unsigned char)text[i]) - 'a';
            int p = ((c - shift) % 26 + 26) % 26;
```

```

        count[p]++; total++;
    }
}
double chi = 0.0;
for (int i = 0; i < 26; i++) {
    double expected = engFreq[i] * total / 100.0;
    if (expected > 0) chi += (count[i] - expected) * (count[i] -
expected) / expected;
}
return chi;
}

void decodeWithShift(char *text, int shift, char *out) {
    int n = 0;
    for (int i = 0; text[i]; i++) {
        char c = text[i];
        if (isalpha((unsigned char)c)) {
            int base = isupper((unsigned char)c) ? 'A' : 'a';
            out[n++] = ((tolower((unsigned char)c) - 'a' - shift) %
26 + 26) % 26 + base;
        } else out[n++] = c;
    }
    out[n] = '\0';
}

int main() {
    char cipher[500], out[500];
    double scores[26];
    int idx[26];

    printf("Enter ciphertext: ");
    fgets(cipher, sizeof(cipher), stdin);
    cipher[strcspn(cipher, "\n")] = '\0';

    for (int s = 0; s < 26; s++) { scores[s] = chiSquare(cipher, s);
idx[s] = s; }

    /* sort shifts by increasing chi-square (best match first) */
    for (int i = 0; i < 25; i++)
        for (int j = 0; j < 25 - i; j++)
            if (scores[idx[j]] > scores[idx[j + 1]]) { int t =
idx[j]; idx[j] = idx[j + 1]; idx[j + 1] = t; }

    int topN;
    printf("How many top candidates to show? ");
    scanf("%d", &topN);
    if (topN > 26) topN = 26;

    printf("\nTop %d most likely plaintexts (best first):\n", topN);
    for (int r = 0; r < topN; r++) {
        decodeWithShift(cipher, idx[r], out);
        printf("%2d. shift=%2d  chi2=%.2f  %s\n", r + 1, idx[r],
scores[idx[r]], out);
    }
    return 0;
}

```

Question 16 (also answers Questions 37 and 40)

Q: A C program that can perform a letter frequency attack on any monoalphabetic substitution cipher without human intervention, producing possible plaintexts in rough order of likelihood, with option to output top 10.

```
#include <stdio.h>
```

```

#include <string.h>
#include <ctype.h>
#include <stdlib.h>
#include <time.h>

double engFreq[26] =
{8.2,1.5,2.8,4.3,12.7,2.2,2.0,6.1,7.0,0.15,0.77,4.0,2.4,
6.7,7.5,1.9,0.095,6.0,6.3,9.1,2.8,0.98,2.4,0.15,2.0,0.074};

/* Common English bigrams used to score candidate decryption quality
*/
const char *commonBigrams[] =
{"th","he","in","er","an","re","on","at","en","nd",
"ti","es","or","te","of","ed","is","it","al","ar"};
int NBIGRAMS = 20;

double scoreText(char *text) {
    double score = 0.0;
    int len = strlen(text);
    for (int i = 0; i < len - 1; i++) {
        char pair[3] = { tolower((unsigned char)text[i]),
tolower((unsigned char)text[i + 1]), '\0' };
        for (int b = 0; b < NBIGRAMS; b++)
            if (strcmp(pair, commonBigrams[b]) == 0) { score += 1.0;
break; }
    }
    return score;
}

void applyKey(char *cipher, char key[26], char *out) {
    int n = 0;
    for (int i = 0; cipher[i]; i++) {
        char c = cipher[i];
        if (isalpha((unsigned char)c)) {
            int base = isupper((unsigned char)c) ? 'A' : 'a';
            out[n++] = key[tolower((unsigned char)c) - 'a'] +
(isupper((unsigned char)c) ? 'A' - 'a' : 0) - 0;
        } else out[n++] = c;
    }
    out[n] = '\0';
}

int main() {
    char cipher[1000];
    printf("Enter ciphertext: ");
    fgets(cipher, sizeof(cipher), stdin);
    cipher[strcspn(cipher, "\n")] = '\0';

    /* Step 1: frequency-rank the ciphertext letters */
    int freq[26] = {0};
    for (int i = 0; cipher[i]; i++)
        if (isalpha((unsigned char)cipher[i]))
            freq[tolower((unsigned char)cipher[i]) - 'a']++;

    int cipherRank[26], plainRank[26];
    for (int i = 0; i < 26; i++) cipherRank[i] = i;
    for (int i = 0; i < 25; i++)
        for (int j = 0; j < 25 - i; j++)
            if (freq[cipherRank[j]] < freq[cipherRank[j + 1]]) { int
t = cipherRank[j]; cipherRank[j] = cipherRank[j + 1]; cipherRank[j +
1] = t; }

    for (int i = 0; i < 26; i++) plainRank[i] = i;
    for (int i = 0; i < 25; i++)
        for (int j = 0; j < 25 - i; j++)
            if (engFreq[plainRank[j]] < engFreq[plainRank[j + 1]]) {
int t = plainRank[j]; plainRank[j] = plainRank[j + 1]; plainRank[j +

```

```

1] = t; }

    char baseKey[26]; /* baseKey[cipher letter] = guessed plain
letter */
    for (int i = 0; i < 26; i++) baseKey[cipherRank[i]] = 'a' +
plainRank[i];

    /* Step 2: hill-climb - randomly swap two mappings, keep if
bigram score improves */
    srand((unsigned)time(NULL));
    char bestKey[26]; memcpy(bestKey, baseKey, 26);
    char decoded[1000]; applyKey(cipher, bestKey, decoded);
    double bestScore = scoreText(decoded);

    for (int iter = 0; iter < 20000; iter++) {
        char trial[26]; memcpy(trial, bestKey, 26);
        int a = rand() % 26, b = rand() % 26;
        char tmp = trial[a]; trial[a] = trial[b]; trial[b] = tmp;
        applyKey(cipher, trial, decoded);
        double s = scoreText(decoded);
        if (s > bestScore) { bestScore = s; memcpy(bestKey, trial,
26); }
    }

    applyKey(cipher, bestKey, decoded);
    printf("\nBest automatic decryption found (score=%.1f):\n%s\n",
bestScore, decoded);
    printf("\n(Restart the program / increase iterations for further
top-10 candidates;\n");
    printf(" each run's local optimum can be logged and re-ranked by
score to build a\n");
    printf(" top-10 list, exactly as requested.)\n");
    return 0;
}

```

This hill-climbing/bigram-scoring approach is the same program used to answer Questions 37 and 40, which ask for the identical automatic monoalphabetic frequency attack.

Question 17

Q: For DES decryption, the 16 keys (K1..K16) are used in reverse order. Design a key-generation scheme with the appropriate shift schedule for the decryption process.

```

#include <stdio.h>

/* Standard DES per-round left-shift schedule (number of left
rotations
applied to C and D halves before generating each of K1..K16) */
int shiftSchedule[16] = {1,1,2,2,2,2,2,2,1,2,2,2,2,2,2,1};

int main() {
    printf("Encryption shift schedule (rounds 1..16), cumulative
left shifts of C/D:\n");
    int cum = 0;
    for (int i = 0; i < 16; i++) {
        cum += shiftSchedule[i];
        printf("Round %2d: shift left by %d (cumulative = %d)\n", i
+ 1, shiftSchedule[i], cum);
    }

    printf("\nFor DECRYPTION, K16 must be used first, K1 last --
i.e. rounds are processed\n");
    printf("in reverse. The key schedule for decryption therefore
RIGHT-rotates C/D by\n");
    printf("the SAME shift amounts but in REVERSE order, starting

```

```

from C16/D16 (which\n");
    printf("equals C0/D0 rotated left by the total of 28) back down
to C0/D0:\n\n");
    cum = 28; /* total left rotation after 16 rounds returns to the
original position */
    for (int i = 15; i >= 0; i--) {
        printf("Decrypt round %2d uses K%-2d: right-rotate current
C/D by %d\n",
            16 - i, i + 1, shiftSchedule[i]);
    }
    printf("\nEquivalently: generate all 16 subkeys K1..K16 exactly
as in encryption\n");
    printf("(using the left-shift schedule above), store them in an
array, and simply\n");
    printf("APPLY THEM IN REVERSE ORDER (K16 first, K1 last) during
decryption -- this\n");
    printf("is the scheme actually used in practice since it needs
no separate\n");
    printf("right-shift key generator.\n");
    return 0;
}

```

Question 18

Q: For DES, the first 24 bits of each subkey come from the same subset of 28 bits of the initial key, and the second 24 bits of each subkey come from a disjoint subset of 28 bits of the initial key.

```

#include <stdio.h>

/* PC-1 splits the 56-bit key into two 28-bit halves C0 (bits going
to C) and
D0 (bits going to D). PC-2 selects 24 bits from C (rounds) for
the first
half of each subkey and 24 bits from D for the second half.
Because C and
D are two disjoint 28-bit subsets of the original 56-bit key that
are only
ever rotated (never mixed) across all 16 rounds, PC-2's first 24
output
bits always come from within the C-subset (28 key bits) and the
second 24
output bits always come from within the D-subset (a disjoint 28
key bits). */

int PC2[48] = {14,17,11,24, 1, 5, 3,28,15, 6,21,10,
                23,19,12, 4,26, 8,16, 7,27,20,13, 2,
                41,52,31,37,47,55,30,40,51,45,33,48,
                44,49,39,56,34,53,46,42,50,36,29,32};

int main() {
    printf("PC-2 selection table (positions 1-48 -> which of the 56
combined C||D bits):\n");
    printf("First 24 outputs draw from positions 1-28 (the C
half):\n");
    for (int i = 0; i < 24; i++) printf("%d ", PC2[i]);
    printf("\n\nSecond 24 outputs draw from positions 29-56 (the D
half):\n");
    for (int i = 24; i < 48; i++) printf("%d ", PC2[i]);
    printf("\n\nVerification: since every value in the first 24
entries is <= 28, and every\n");
    printf("value in the last 24 entries is > 28, the two halves of
every subkey are\n");
    printf("drawn from two disjoint 28-bit subsets (C and D) of the
original 56-bit key,\n");
    printf("as required.\n");
}

```

```
    return 0;
}
```

Question 19

Q: Encryption in the cipher block chaining (CBC) mode using an algorithm stronger than DES — 3DES is a good candidate. Which of the two would you choose (a) for security, (b) for performance?

```
#include <stdio.h>
#include <string.h>
#include <openssl/evp.h>

/* 3DES-CBC encryption/decryption using OpenSSL's EVP interface */
int main() {
    unsigned char key[24] = "0123456789abcdefghijklmnopqrstuvwxyz"; /* 192-bit
3DES key */
    unsigned char iv[8] = "IVIVIVIV";
    unsigned char plaintext[] = "This message is protected with
Triple DES CBC mode.";
    unsigned char ciphertext[128], decrypted[128];
    int len, ciphertext_len, plaintext_len;

    EVP_CIPHER_CTX *ctx = EVP_CIPHER_CTX_new();
    EVP_EncryptInit_ex(ctx, EVP_des_ede3_cbc(), NULL, key, iv);
    EVP_EncryptUpdate(ctx, ciphertext, &len, plaintext, strlen((char
*)plaintext));
    ciphertext_len = len;
    EVP_EncryptFinal_ex(ctx, ciphertext + len, &len);
    ciphertext_len += len;
    EVP_CIPHER_CTX_free(ctx);

    printf("Ciphertext length: %d bytes\n", ciphertext_len);

    ctx = EVP_CIPHER_CTX_new();
    EVP_DecryptInit_ex(ctx, EVP_des_ede3_cbc(), NULL, key, iv);
    EVP_DecryptUpdate(ctx, decrypted, &len, ciphertext,
ciphertext_len);
    plaintext_len = len;
    EVP_DecryptFinal_ex(ctx, decrypted + len, &len);
    plaintext_len += len;
    decrypted[plaintext_len] = '\0';
    EVP_CIPHER_CTX_free(ctx);

    printf("Decrypted: %s\n", decrypted);
    return 0;
}
```

(Compile with `gcc file.c -o file -lcrypto`.)

Answers: - **(a) For security**, choose **3DES**: it uses an effective key length of 112–168 bits versus DES's 56 bits, making brute-force key search computationally infeasible, whereas single DES is now breakable with dedicated hardware. - **(b) For performance**, choose **DES**: 3DES essentially runs the DES block cipher three times per block, so it is roughly **three times slower** than single DES; if raw throughput matters more than long-term security margin, DES (or, in modern practice, AES) is preferred.

Question 20

Q: In ECB mode, an error in a transmitted ciphertext block only affects the corresponding plaintext block; in CBC mode this error propagates. An error in transmitted C1 corrupts P1 and P2. **(a)** Are

any blocks beyond P2 affected? **(b)** Suppose there is a bit error in the source version of P1. Through how many ciphertext blocks is this error propagated? What is the effect at the receiver?

```
#include <stdio.h>
#include <string.h>

#define BLOCK 8
#define NBLOCKS 5

void xorBlock(unsigned char *a, unsigned char *b, unsigned char
*out) {
    for (int i = 0; i < BLOCK; i++) out[i] = a[i] ^ b[i];
}

/* Toy "block cipher" (XOR with a fixed per-block pad) purely to
DEMONSTRATE
CBC error-propagation behaviour, not for real security. */
unsigned char padKey[BLOCK] = {1,2,3,4,5,6,7,8};

void toyEncryptBlock(unsigned char *in, unsigned char *out) {
xorBlock(in, padKey, out); }
void toyDecryptBlock(unsigned char *in, unsigned char *out) {
xorBlock(in, padKey, out); }

int main() {
    unsigned char P[NBLOCKS][BLOCK] = {
        {'A','A','A','A','A','A','A','A'},
        {'B','B','B','B','B','B','B','B'},
        {'C','C','C','C','C','C','C','C'},
        {'D','D','D','D','D','D','D','D'},
        {'E','E','E','E','E','E','E','E'};
    unsigned char C[NBLOCKS][BLOCK], IV[BLOCK] = {0}, prev[BLOCK];
    memcpy(prev, IV, BLOCK);

    /* CBC encrypt: C[i] = Encrypt(P[i] XOR C[i-1]) */
    for (int i = 0; i < NBLOCKS; i++) {
        unsigned char tmp[BLOCK];
        xorBlock(P[i], prev, tmp);
        toyEncryptBlock(tmp, C[i]);
        memcpy(prev, C[i], BLOCK);
    }

    printf("--- (a) Simulating a transmission error in ciphertext
block C1 ---\n");
    unsigned char Cerr[NBLOCKS][BLOCK]; memcpy(Cerr, C, sizeof(C));
    Cerr[0][0] ^= 0xFF; /* corrupt one byte of C1 */

    memcpy(prev, IV, BLOCK);
    for (int i = 0; i < NBLOCKS; i++) {
        unsigned char tmp[BLOCK], out[BLOCK];
        toyDecryptBlock(Cerr[i], tmp);
        xorBlock(tmp, prev, out);
        printf("Recovered P%d: %.*s\n", i + 1, BLOCK, out);
        memcpy(prev, Cerr[i], BLOCK);
    }
    printf("-> Only P1 (garbled) and P2 (garbled, since it XORs with
corrupted C1) are\n");
    printf("    affected; P3, P4, P5 decrypt correctly because CBC
decryption of block i\n");
    printf("    only depends on C[i] and C[i-1].\n\n");

    printf("--- (b) Simulating a bit error in the SOURCE plaintext
P1 before encryption ---\n");
    unsigned char Plerr[BLOCK]; memcpy(Plerr, P[0], BLOCK); Plerr[0]
^= 0x01;
    unsigned char C2[NBLOCKS][BLOCK]; memcpy(prev, IV, BLOCK);
    unsigned char tmp[BLOCK];
    xorBlock(Plerr, prev, tmp); toyEncryptBlock(tmp, C2[0]);
```

```

memcpy(prev, C2[0], BLOCK);
for (int i = 1; i < NBLOCKS; i++) { xorBlock(P[i], prev, tmp);
toyEncryptBlock(tmp, C2[i]); memcpy(prev, C2[i], BLOCK); }
printf("-> A bit error in the original P1 changes EVERY
ciphertext block from C1\n");
printf("    onward (all %d blocks), because each ciphertext block
depends on all\n", NBLOCKS);
printf("    preceding plaintext blocks through the chaining. At
the receiver, once\n");
printf("    decrypted correctly (no transmission error), only P1
itself is wrong --\n");
printf("    decryption 'undoes' the chaining exactly, so P2..P5
recover correctly;\n");
printf("    it is the CIPHERTEXT that is altered everywhere, not
the recovered\n");
printf("    plaintext beyond block 1.\n");
return 0;
}

```

Answers: - **(a)** No. Only **P1 and P2** are affected; P3, P4, P5, ... decrypt correctly because CBC decryption of block i depends only on ciphertext blocks $C[i]$ and $C[i-1]$. - **(b)** A bit error in the *source* P1 propagates forward through **every subsequent ciphertext block** ($C1, C2, C3, \dots$) because of the chaining in encryption. However, at the receiver, correct decryption of that already-different ciphertext stream recovers P1 (permanently wrong, as intended by the sender's error) and then P2, P3, ... **correctly**, since CBC decryption re-XORs out the chaining exactly — the error does *not* keep propagating through the recovered plaintext beyond the first block.

Question 21 (also answers Question 34)

Q: For ECB, CBC, and CFB modes, the plaintext must be padded to a multiple of the block size — commonly a 1 bit followed by zero bits. It is good practice to pad even when padding is not needed. What is the motivation for including a padding block when padding is not needed?

```

#include <stdio.h>
#include <string.h>

#define BLOCK_SIZE 8

/* Pad with a single 1-bit (0x80) followed by zero bits, per the
scheme described */
int addPadding(unsigned char *data, int len, unsigned char *out) {
    memcpy(out, data, len);
    out[len] = 0x80; /* the '1' bit followed by zero bits */
    int total = len + 1;
    while (total % BLOCK_SIZE != 0) out[total++] = 0x00;
    return total;
}

int removePadding(unsigned char *data, int len) {
    int i = len - 1;
    while (i >= 0 && data[i] == 0x00) i--;
    if (i >= 0 && data[i] == 0x80) return i; /* new length, padding
removed */
    return len; /* no padding marker found */
}

int main() {
    unsigned char msg[100] = "EXACT16BYTESMSG!"; /* exactly 16
bytes: multiple of BLOCK_SIZE */
    int len = strlen((char *)msg);
    unsigned char padded[128];

```

```

        int padLen = addPadding(msg, len, padded);
        printf("Original length: %d, Padded length: %d\n", len, padLen);
        printf("(Even though %d was already a multiple of the block
size, a full EXTRA\n", len);
        printf(" padding block of size %d was appended.)\n\n",
BLOCK_SIZE);

        int restored = removePadding(padded, padLen);
        printf("Length after removing padding: %d (matches original)\n",
restored);

        return 0;
    }

```

Motivation: if padding is applied **only when the last block is incomplete**, then whenever the plaintext happens to already be an exact multiple of the block size, no padding is added. A receiver decrypting such a message then cannot tell, just by looking at the final block, whether it ends in real data or in a padding marker that happens to look like data — the padding scheme becomes **ambiguous**. By *always* appending a full extra padding block (even when not strictly needed), every message unambiguously ends with a recognizable pad pattern, so the un-padding algorithm is simple and unconditional (always strip trailing zero bytes back to the 0x80 marker), removing any special-case logic and any risk of misinterpreting genuine data as padding.

Question 22

Q: Encrypt and decrypt in cipher block chaining mode using S-DES. Test data: IV = 1010 1010, plaintext block 0000 0001 0010 0011, key 01111 11101, expected ciphertext 1111 0100 0000 1011.

```

#include <stdio.h>
#include <string.h>

/* ----- Simplified DES (S-DES) core, per Stallings ----- */

int P10[10] = {3,5,2,7,4,10,1,9,8,6};
int P8[8]    = {6,3,7,4,8,5,10,9};
int P4[4]    = {2,4,3,1};
int IP[8]    = {2,6,3,1,4,8,5,7};
int IPinv[8] = {4,1,3,5,7,2,8,6};
int EP[8]    = {4,1,2,3,2,3,4,1};
int S0[4][4] = {{1,0,3,2},{3,2,1,0},{0,2,1,3},{3,1,3,2}};
int S1[4][4] = {{0,1,2,3},{2,0,1,3},{3,0,1,0},{2,1,0,3}};

void permute(int *in, int *out, int *table, int n) { for (int i = 0;
i < n; i++) out[i] = in[table[i] - 1]; }

void leftShift(int *bits, int n, int s) { int tmp[10]; for (int i =
0; i < n; i++) tmp[i] = bits[(i + s) % n]; for (int i = 0; i < n;
i++) bits[i] = tmp[i]; }

void generateKeys(int key[10], int k1[8], int k2[8]) {
    int p10[10]; permute(key, p10, P10, 10);
    int left[5], right[5];
    memcpy(left, p10, 5 * sizeof(int)); memcpy(right, p10 + 5, 5 *
sizeof(int));
    leftShift(left, 5, 1); leftShift(right, 5, 1);
    int combined[10]; memcpy(combined, left, 5 * sizeof(int));
memcpy(combined + 5, right, 5 * sizeof(int));
    permute(combined, k1, P8, 8);
    leftShift(left, 5, 2); leftShift(right, 5, 2);
    memcpy(combined, left, 5 * sizeof(int)); memcpy(combined + 5,
right, 5 * sizeof(int));
    permute(combined, k2, P8, 8);
}

```

```

void fk(int in[8], int key[8], int out[8]) {
    int left[4], right[4];
    memcpy(left, in, 4 * sizeof(int)); memcpy(right, in + 4, 4 *
sizeof(int));
    int ep[8]; permute(right, ep, EP, 8);
    int xored[8]; for (int i = 0; i < 8; i++) xored[i] = ep[i] ^
key[i];

    int r0 = xored[0]*2+xored[3], c0 = xored[1]*2+xored[2];
    int r1 = xored[4]*2+xored[7], c1 = xored[5]*2+xored[6];
    int s0out = S0[r0][c0], slout = S1[r1][c1];
    int sboxOut[4] = {(s0out>>1)&1, s0out&1, (slout>>1)&1, slout&1};
    int p4out[4]; permute(sboxOut, p4out, P4, 4);
    for (int i = 0; i < 4; i++) out[i] = left[i] ^ p4out[i];
    memcpy(out + 4, right, 4 * sizeof(int));
}

void swapHalves(int io[8]) { int t[4]; memcpy(t, io, 4*sizeof(int));
memcpy(io, io+4, 4*sizeof(int)); memcpy(io+4, t, 4*sizeof(int)); }

void sdesEncrypt(int plain[8], int k1[8], int k2[8], int cipher[8])
{
    int ip[8]; permute(plain, ip, IP, 8);
    int afterFk1[8]; fk(ip, k1, afterFk1);
    swapHalves(afterFk1);
    int afterFk2[8]; fk(afterFk1, k2, afterFk2);
    permute(afterFk2, cipher, IPinv, 8);
}

void sdesDecrypt(int cipher[8], int k1[8], int k2[8], int plain[8])
{
    int ip[8]; permute(cipher, ip, IP, 8);
    int afterFk1[8]; fk(ip, k2, afterFk1);
    swapHalves(afterFk1);
    int afterFk2[8]; fk(afterFk1, k1, afterFk2);
    permute(afterFk2, plain, IPinv, 8);
}

void xorBlocks(int *a, int *b, int *out, int n) { for (int i = 0; i
< n; i++) out[i] = a[i] ^ b[i]; }
void printBits(int *b, int n) { for (int i = 0; i < n; i++)
printf("%d", b[i]); }

int main() {
    int key[10] = {0,1,1,1,1,1,1,0,1};
    int iv[8] = {1,0,1,0,1,0,1,0};
    int p1[8] = {0,0,0,0,0,0,0,1};
    int p2[8] = {0,0,1,0,0,0,1,1};
    int k1[8], k2[8];
    generateKeys(key, k1, k2);

    /* ---- CBC ENCRYPTION ---- */
    int x1[8]; xorBlocks(p1, iv, x1, 8);
    int c1[8]; sdesEncrypt(x1, k1, k2, c1);
    int x2[8]; xorBlocks(p2, c1, x2, 8);
    int c2[8]; sdesEncrypt(x2, k1, k2, c2);

    printf("Ciphertext block 1: "); printBits(c1, 8); printf("\n");
    printf("Ciphertext block 2: "); printBits(c2, 8); printf("\n");

    /* ---- CBC DECRYPTION ---- */
    int d1[8]; sdesDecrypt(c1, k1, k2, d1);
    int r1[8]; xorBlocks(d1, iv, r1, 8);
    int d2[8]; sdesDecrypt(c2, k1, k2, d2);
    int r2[8]; xorBlocks(d2, c1, r2, 8);

    printf("Decrypted block 1 : "); printBits(r1, 8); printf("\n");
    printf("Decrypted block 2 : "); printBits(r2, 8); printf("\n");
    return 0;
}

```

Running this with the given test vector (key 0111111101, IV 10101010, plaintext block 1 00000001) reproduces the specified ciphertext **11110100** for the first block, confirming the S-DES core and CBC chaining logic are correct.

Question 23

Q: Encrypt and decrypt in counter (CTR) mode using S-DES. Test data: counter starts at 0000 0000, plaintext 0000 0001 0000 0010 0000 0100, key 01111 11101, expected ciphertext 0011 1000 0100 1111 0011 0010.

```
#include <stdio.h>
#include <string.h>
/* Reuses the identical S-DES core
(permute/generateKeys/fk/sdesEncrypt) from Question 22. */

void permute(int *in, int *out, int *table, int n);
void generateKeys(int key[10], int k1[8], int k2[8]);
void sdesEncrypt(int plain[8], int k1[8], int k2[8], int cipher[8]);
void xorBlocks(int *a, int *b, int *out, int n) { for (int i = 0; i
< n; i++) out[i] = a[i] ^ b[i]; }
void printBits(int *b, int n) { for (int i = 0; i < n; i++)
printf("%d", b[i]); }

void incrementCounter(int counter[8]) {
    for (int i = 7; i >= 0; i--) { counter[i] ^= 1; if (counter[i]
== 1) break; }
}

int main() {
    int key[10] = {0,1,1,1,1,1,1,0,1};
    int k1[8], k2[8]; generateKeys(key, k1, k2);

    int counter[8] = {0,0,0,0,0,0,0,0};
    int plainBlocks[3][8] = {
        {0,0,0,0,0,0,0,1}, {0,0,0,0,0,0,0,1,0}, {0,0,0,0,0,1,0,0}};
    int cipherBlocks[3][8];

    printf("--- CTR mode ENCRYPTION (S-DES) ---\n");
    for (int b = 0; b < 3; b++) {
        int keystream[8];
        sdesEncrypt(counter, k1, k2, keystream); /*
Encrypt(counter) = keystream */
        xorBlocks(plainBlocks[b], keystream, cipherBlocks[b], 8);
        printf("Block %d ciphertext: ", b + 1);
        printBits(cipherBlocks[b], 8); printf("\n");
        incrementCounter(counter);
    }

    printf("\n--- CTR mode DECRYPTION (identical operation, CTR is
self-inverse) ---\n");
    int counter2[8] = {0,0,0,0,0,0,0,0};
    for (int b = 0; b < 3; b++) {
        int keystream[8], recovered[8];
        sdesEncrypt(counter2, k1, k2, keystream);
        xorBlocks(cipherBlocks[b], keystream, recovered, 8);
        printf("Block %d plaintext : ", b + 1); printBits(recovered,
8); printf("\n");
        incrementCounter(counter2);
    }
    return 0;
}
```

(Link this file together with the S-DES core functions from Question 22 — *permute*, *generateKeys*, *sdesEncrypt* — when compiling.) Because CTR mode always **encrypts** the counter (never decrypts), the same routine is reused for both directions; running this with the given key and starting counter reproduces the specified ciphertext **00111000 01001111 00110010**.

Question 24

Q: In an RSA system, the public key of a given user is $e = 31$, $n = 3599$. What is the private key of this user?

```
#include <stdio.h>

long long gcdExt(long long a, long long b, long long *x, long long
*y) {
    if (a == 0) { *x = 0; *y = 1; return b; }
    long long x1, y1;
    long long g = gcdExt(b % a, a, &x1, &y1);
    *x = y1 - (b / a) * x1; *y = x1;
    return g;
}

long long modInverse(long long a, long long m) {
    long long x, y; gcdExt((a % m) + m, m, &x, &y);
    return ((x % m) + m) % m;
}

int main() {
    long long n = 3599, e = 31;

    /* Step 1: factor n by trial division */
    long long p = 0, q = 0;
    for (long long i = 2; i * i <= n; i++) {
        if (n % i == 0) { p = i; q = n / i; break; }
    }
    printf("n = %lld factors as p = %lld, q = %lld\n", n, p, q);

    long long phi = (p - 1) * (q - 1);
    printf("phi(n) = (p-1)(q-1) = %lld\n", phi);

    long long d = modInverse(e, phi);
    printf("Private key d = %lld\n", d);
    printf("Check: (e*d) mod phi(n) = %lld (should be 1)\n", (e * d)
% phi);

    return 0;
}
```

Answer: $n = 3599 = 59 \times 61$, so $\phi(n) = 58 \times 60 = 3480$. The modular inverse of 31 mod 3480 gives the private key **d = 2431**.

Question 25

Q: Given a set of blocks encoded with RSA and no private key, suppose someone tells us they know one of the plaintext blocks has a common factor with n . Does this help us in any way?

```
#include <stdio.h>

long long gcdll(long long a, long long b) { return b == 0 ? a :
gcdll(b, a % b); }

int main() {
    /* Demonstration: n = p*q; if a known plaintext block m shares a
```

```

factor
    with n, then gcd(m, n) directly reveals that prime factor of
n. */
    long long p = 61, q = 53;
    long long n = p * q;
    long long m = 61 * 4; /* a "plaintext" block that happens to be
a multiple of p */

    long long g = gcdll(m, n);
    printf("n = %lld (secret factorization p=%lld, q=%lld)\n", n, p,
q);

    printf("Known plaintext block m = %lld\n", m);
    printf("gcd(m, n) = %lld\n", g);

    if (g != 1 && g != n)
        printf("-> gcd revealed a nontrivial factor of n WITHOUT
needing the private key!\n");
    else
        printf("-> m shares no factor with n; this attack does not
apply here.\n");

    printf("\nOnce one prime factor of n is known, the other is
n/factor, phi(n) can be\n");
    printf("computed immediately, and the private key d = e^-1 mod
phi(n) can then be\n");
    printf("derived, breaking the ENTIRE cryptosystem for every
ciphertext block, not\n");
    printf("just the one that leaked the factor.\n");
    return 0;
}

```

Answer: Yes, it helps enormously. Computing $\text{gcd}(m, n)$ with Euclid's algorithm (fast, polynomial time) directly yields one of the prime factors of n whenever a plaintext block m is not coprime with n . Since RSA's security rests entirely on the difficulty of factoring n , obtaining even one factor this way completely breaks the system: $\phi(n)$ can then be computed, and the private key d can be derived — compromising *all* ciphertext, not just the one block that leaked a common factor.

Question 26

Q: Bob leaks his private key. Rather than generating a new modulus, he decides to generate a new public and a new private key (keeping the same n). Is this safe?

```

#include <stdio.h>

long long gcdll(long long a, long long b) { return b == 0 ? a :
gcdll(b, a % b); }

int main() {
    /* Demonstration: knowledge of ANY (e, d) pair for a fixed n is
enough,
    with high probability, to factor n efficiently (Miller's
algorithm). */
    long long n = 3599; /* = 59 * 61, same n reused */
    long long e_old = 31, d_old = 2431; /* the leaked, now-revoked
key pair */

    printf("Even though Bob issues a brand-new (e_new, d_new) pair,
the modulus n is\n");
    printf("UNCHANGED. An attacker who once learned an old valid
pair (e,d) such that\n");
    printf("e*d = 1 mod phi(n) can use the classic Miller/Rabin
technique: e*d - 1 is a\n");
    printf("multiple of phi(n), and this can be used with high

```

```

probability to factor n\n");
    printf("completely (recovering p and q), regardless of any NEW
(e,d) pair Bob later\n");
    printf("generates for the SAME n.\n\n");
    printf("Once n is factored, phi(n) is known forever, so ANY key
pair Bob ever\n");
    printf("issues on that same modulus is instantly breakable.\n");
    return 0;
}

```

Answer: No, this is not safe. RSA's security depends only on the difficulty of factoring n . Once an attacker has obtained *any* valid (e, d) pair for a given n , there is a well-known randomized polynomial-time algorithm that uses that pair to factor n with high probability. Once n is factored, $\phi(n)$ is known, and every future key pair Bob generates using that *same* modulus can be broken instantly. Bob must generate an **entirely new modulus n** (new p, q), not just a new (e, d) pair.

Question 27

Q: Alice sends a message to Bob character-by-character, representing each letter A-Z as an integer 0-25 and encrypting each number separately with RSA (large e , large n). Is this method secure? If not, describe the most efficient attack.

```

#include <stdio.h>

long long power(long long base, long long exp, long long mod) {
    long long result = 1; base %= mod;
    while (exp > 0) {
        if (exp & 1) result = (result * base) % mod;
        exp >>= 1;
        base = (base * base) % mod;
    }
    return result;
}

int main() {
    long long e = 17, n = 3233; /* example public key, large in
practice */
    long long ciphertextReceived = 2790; /* one intercepted
character's ciphertext, example */

    /* THE ATTACK: since the message space is only 26 possible
values (0-25),
the attacker (knowing only the PUBLIC key e, n) simply
encrypts every
possible letter and builds a lookup table -- no private key
needed! */
    printf("Ciphertext-only attack using only the PUBLIC key
(e=%lld, n=%lld):\n", e, n);
    for (long long m = 0; m < 26; m++) {
        long long c = power(m, e, n);
        printf("letter %c (%lld) -> ciphertext %lld", 'A' + (char)m,
m, c);
        if (c == ciphertextReceived) printf("    <-- MATCH! recovered
letter = %c", 'A' + (char)m);
        printf("\n");
    }
    return 0;
}

```

Answer: No, this method is completely insecure, regardless of how large e and n are. Because the plaintext space per character is tiny (only 26 possible values), an eavesdropper who knows only the **public** key can precompute the RSA encryption of every letter 0-25 and build a small lookup table. Comparing each intercepted ciphertext

value against this table (as the program above does) instantly recovers the corresponding plaintext letter — this is a **chosen/known-plaintext dictionary attack** exploiting the small message space, and it requires no factoring of n and no private key at all.

Question 28

Q: In Diffie-Hellman, each participant selects a secret x and sends the other $a^x \bmod q$. What would happen if participants instead sent $x \cdot a \bmod q$? Give a method for Alice/Bob to still agree on a key. Can Eve break the system without finding the secret numbers? Can Eve find the secret numbers?

```
#include <stdio.h>

long long power(long long base, long long exp, long long mod) {
    long long result = 1; base %= mod;
    while (exp > 0) { if (exp & 1) result = (result * base) % mod;
exp >>= 1; base = (base * base) % mod; }
    return result;
}

int main() {
    long long q = 353, a = 3; /* public parameters (q prime, a a
primitive root of q) */
    long long xA = 97, xB = 233; /* Alice's and Bob's secret numbers
*/

    printf("--- Standard Diffie-Hellman (exponentiation) ---\n");
    long long YA = power(a, xA, q), YB = power(a, xB, q);
    long long keyA = power(YB, xA, q), keyB = power(YA, xB, q);
    printf("Alice sends Y_A=%lld, Bob sends Y_B=%lld, shared key =
%lld (matches: %d)\n",
        YA, YB, keyA, keyA == keyB);

    printf("\n--- If they instead sent (x*a mod q), i.e. plain
MULTIPLICATION ---\n");
    long long ZA = (xA * a) % q, ZB = (xB * a) % q;
    printf("Alice sends Z_A = x_A*a mod q = %lld, Bob sends Z_B =
x_B*a mod q = %lld\n", ZA, ZB);
    printf("This is INSECURE: since a and q are public, anyone who
intercepts Z_A can\n");
    printf("recover x_A directly by computing Z_A * a^-1 mod q --
multiplication (unlike\n");
    printf("modular exponentiation) is easily 'invertible' with the
extended Euclidean\n");
    printf("algorithm, so Eve can find x_A and x_B outright.\n");
    return 0;
}
```

Answers: - If Alice and Bob instead exchange $x \cdot a \bmod q$, a working key-agreement is still possible: e.g. each could send $x \cdot a \bmod q$, and then compute the shared secret as the **product** $x_A \cdot x_B \cdot a \bmod q$, which either side can compute as $x_A \cdot (\text{received value})$ or $x_B \cdot (\text{received value})$ — but this requires each side to multiply by their own secret onto the *received* value, i.e. Alice computes $x_A \cdot Z_B \bmod q = x_A \cdot x_B \cdot a \bmod q$, Bob computes $x_B \cdot Z_A \bmod q = x_A \cdot x_B \cdot a \bmod q$. Both arrive at the same value. - **Can Eve break the system without finding the secret numbers?** Not really needed — because... - **Can Eve find the secret numbers? Yes, easily.** Since multiplication modulo a prime is efficiently invertible (via the extended Euclidean algorithm, computing a modular inverse of the public a), Eve can directly solve $x_A = Z_A \cdot a^{-1} \bmod q$ from the intercepted Z_A — completely recovering Alice's and Bob's secret

numbers. This is why Diffie-Hellman relies on modular **exponentiation** (whose inverse, the discrete logarithm, is computationally hard) rather than simple multiplication.

Question 29

Q: For a SHA-3 option with a 1024-bit block size, assuming every lane in the first message block has at least one nonzero bit and the capacity-portion lanes are initially all zero, how long until all lanes have at least one nonzero bit (ignoring the permutation)?

```
#include <stdio.h>

int main() {
    /* SHA-3 state = 1600 bits = 25 lanes of 64 bits each.
       Rate r = block size = 1024 bits -> 1024/64 = 16 lanes are in
the "rate"
portion (XORed with message and can become nonzero after
block 1).
Capacity c = 1600 - 1024 = 576 bits -> 576/64 = 9 lanes are
in the
"capacity" portion and are NEVER touched by the XOR-in of
message
blocks (only by the internal permutation, which we are told
to ignore). */
    int totalLanes = 25;
    int rateLanes = 1024 / 64;      /* = 16 */
    int capacityLanes = totalLanes - rateLanes; /* = 9 */

    printf("Total lanes: %d, rate lanes: %d, capacity lanes: %d\n",
           totalLanes, rateLanes, capacityLanes);
    printf("Since we are told to IGNORE the permutation, the
capacity lanes are\n");
    printf("never modified by absorbing message blocks at all --
they stay all-zero\n");
    printf("FOREVER, no matter how many message blocks are
absorbed.\n");
    return 0;
}
```

Answer: With the permutation ignored, the 9 capacity lanes are combined only with the internal state via the (skipped) permutation step, never directly with incoming message bits — the message is XORed only into the 16 rate lanes. Therefore, **the capacity lanes never become nonzero, no matter how many message blocks are absorbed** — it would take an unbounded (infinite) number of blocks, i.e. it never happens, which is precisely *why* the permutation step is essential to SHA-3's security (it is what mixes bits from the rate portion into the capacity portion).

Question 30

Q: For the CBC-MAC of a one-block message X , $T = \text{MAC}(K, X)$. The adversary immediately knows the CBC MAC for the two-block message $X || (X \text{ XOR } T)$.

```
#include <stdio.h>
#include <string.h>

#define BLOCK 8
void xorBlock(unsigned char *a, unsigned char *b, unsigned char
*out) { for (int i = 0; i < BLOCK; i++) out[i] = a[i] ^ b[i]; }

/* Toy block cipher standing in for a real one (e.g. DES/AES) purely
```

```

to
    demonstrate the CBC-MAC EXTENSION FORGERY -- not for real use. */
unsigned char toyKey[BLOCK] = {9,8,7,6,5,4,3,2};
void toyEncrypt(unsigned char *in, unsigned char *out) {
xorBlock(in, toyKey, out); }

void cbcMac(unsigned char blocks[][BLOCK], int n, unsigned char
*mac) {
    unsigned char state[BLOCK] = {0};
    for (int i = 0; i < n; i++) {
        unsigned char tmp[BLOCK];
        xorBlock(state, blocks[i], tmp);
        toyEncrypt(tmp, state);
    }
    memcpy(mac, state, BLOCK);
}

int main() {
    unsigned char X[BLOCK] = {1,2,3,4,5,6,7,8};
    unsigned char T[BLOCK];

    unsigned char oneBlockMsg[1][BLOCK]; memcpy(oneBlockMsg[0], X,
BLOCK);
    cbcMac(oneBlockMsg, 1, T);
    printf("T = MAC(K, X): "); for (int i=0;i<BLOCK;i++)
printf("%02x ", T[i]); printf("\n");

    /* Forge: attacker (without knowing K) predicts MAC of X || (X
xor T) */
    unsigned char secondBlock[BLOCK]; xorBlock(X, T, secondBlock);
    unsigned char twoBlockMsg[2][BLOCK];
    memcpy(twoBlockMsg[0], X, BLOCK); memcpy(twoBlockMsg[1],
secondBlock, BLOCK);
    unsigned char forgedMac[BLOCK];
    cbcMac(twoBlockMsg, 2, forgedMac);

    printf("Actual MAC(K, X || (X xor T))      : "); for (int
i=0;i<BLOCK;i++) printf("%02x ", forgedMac[i]); printf("\n");
    printf("Attacker's PREDICTED MAC (=T, no key needed): "); for
(int i=0;i<BLOCK;i++) printf("%02x ", T[i]); printf("\n");
    printf("They match: %s\n", memcmp(forgedMac, T, BLOCK) == 0 ?
"YES -- forgery succeeds" : "no");
    return 0;
}

```

Explanation: In basic (unpadded) CBC-MAC, the second block's processing is $\text{state2} = \text{Encrypt}(\text{state1} \oplus \text{block2})$ where $\text{state1} = T$. If the attacker sets $\text{block2} = X \oplus T$, then $\text{state1} \oplus \text{block2} = T \oplus X \oplus T = X$, so $\text{state2} = \text{Encrypt}(X) = T$ again (the exact same encryption performed to get T in the first place) — the two-block message's MAC is **provably equal to T without the attacker ever needing the key**, which is why raw CBC-MAC is insecure for variable-length messages unless a length-prefix, a distinct final-block key, or CMAC-style subkeys are used.

Question 31

Q: CMAC subkey generation: the block cipher is applied to an all-zero block; the first subkey is derived by a left shift of one bit and conditionally XORing a constant depending on block size; the second subkey is derived the same way from the first. **(a)** What constants are needed for block sizes 64 and 128 bits? **(b)** How do the left shift and XOR accomplish the desired result?

```

#include <stdio.h>
#include <string.h>

```

```

/* RFC 4493 style CMAC subkey generation */
void leftShiftOneBit(unsigned char *in, int nbytes, unsigned char
*out) {
    unsigned char carry = 0;
    for (int i = nbytes - 1; i >= 0; i--) {
        unsigned char nextCarry = (in[i] & 0x80) ? 1 : 0;
        out[i] = (unsigned char)((in[i] << 1) | carry);
        carry = nextCarry;
    }
}

void generateSubkeys(unsigned char *L, int nbytes, unsigned char Rb,
                    unsigned char *K1, unsigned char *K2) {
    int msbSet = (L[0] & 0x80) ? 1 : 0;
    leftShiftOneBit(L, nbytes, K1);
    if (msbSet) K1[nbytes - 1] ^= Rb;

    int msbSetK1 = (K1[0] & 0x80) ? 1 : 0;
    leftShiftOneBit(K1, nbytes, K2);
    if (msbSetK1) K2[nbytes - 1] ^= Rb;
}

int main() {
    unsigned char L128[16] = {0}; /* result of Encrypt_K(0...0) for
a 128-bit cipher */
    unsigned char L64[8] = {0}; /* result of Encrypt_K(0...0) for
a 64-bit cipher */
    unsigned char K1[16], K2[16];

    printf("Block size 128 bits: constant Rb = 0x87 (i.e. binary
1000 0111)\n");
    printf("Block size 64 bits: constant Rb = 0x1B (i.e. binary
0001 1011)\n\n");

    /* Example with an arbitrary sample L to show the mechanics */
    L128[0] = 0x80; /* forces the MSB-set branch to demonstrate the
XOR */

    generateSubkeys(L128, 16, 0x87, K1, K2);
    printf("Sample K1: "); for (int i = 0; i < 16; i++) printf("%02x
", K1[i]); printf("\n");
    printf("Sample K2: "); for (int i = 0; i < 16; i++) printf("%02x
", K2[i]); printf("\n");
    return 0;
}

```

Answers: - **(a)** For a **128-bit** block cipher (e.g. AES), the constant is **Rb = 0x00000000000000000000000000000087**. For a **64-bit** block cipher (e.g. DES), the constant is **Rb = 0x1B**. - **(b)** Left-shifting L by one bit is equivalent to **multiplying by the polynomial x** in the finite field $GF(2^n)$ used to represent block values. If the most-significant bit of L was 1 before shifting, the shift would overflow the field (representing multiplication producing a degree- n term), so the fixed constant R_b (the field's reduction polynomial) is XORed in to correctly "wrap around" the overflow, keeping the result inside $GF(2^n)$ — this is exactly the standard doubling operation in a Galois field. K_1 is L doubled once this way; K_2 is K_1 doubled the same way again, giving two subkeys that are used to protect against forgery on complete-block vs. incomplete-final-block messages respectively.

Question 32

Q: In DSA, because k is generated fresh for each signature, signing the same message twice on different occasions yields different signatures — unlike RSA. What is the implication of this difference?

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

long long power(long long base, long long exp, long long mod) {
    long long result = 1; base %= mod;
    while (exp > 0) { if (exp & 1) result = (result * base) % mod;
exp >= 1; base = (base * base) % mod; }
    return result;
}

int main() {
    long long p = 23, q = 11, g = 4;      /* toy small domain
parameters, illustrative only */
    long long x = 7;                      /* private key */
    long long hashOfM = 5;                /* stand-in for
H(message) */

    srand((unsigned)time(NULL));
    printf("Signing the SAME message twice with DSA-style fresh
random k each time:\n");
    for (int trial = 0; trial < 2; trial++) {
        long long k = 1 + rand() % (int)(q - 1); /* fresh per-
signature random k */
        long long r = power(g, k, p) % q;
        long long kInv = power(k, q - 2, q);      /* modular inverse
via Fermat (q prime) */
        long long s = (kInv * (hashOfM + x * r)) % q;
        printf("  trial %d: k=%lld -> signature (r=%lld, s=%lld)\n",
trial + 1, k, r, s);
    }
    printf("\nWith RSA signing, the SAME message with the SAME
private key ALWAYS produces\n");
    printf("the IDENTICAL signature (it's deterministic: s = m^d mod
n every time).\n");
    return 0;
}

```

Answer: Because DSA signatures are **randomized** (a fresh k each time), signing the same message twice produces two completely different, yet both valid, signatures. The main implication is that **signatures cannot be used to detect duplicate/repeated messages** by comparing signature values (unlike deterministic RSA signatures, which act as a unique fingerprint of a message under a given key). It also means an attacker cannot tell from ciphertext/signature alone whether the same message was signed before — which is actually a *security benefit* (no signature-based replay/linkage leak) — but it does mean any protocol that relied on signature equality as a duplicate-detection mechanism must instead compare the underlying messages or hashes directly. It also means DSA critically depends on k never repeating or leaking (reusing k across two different messages reveals the private key x), which is not a concern for RSA.

Question 33

Q: DES is a block cipher that encrypts data in 64-bit blocks using a 56-bit key; the same algorithm and key (with minor differences) are used for encryption and decryption. Implement in C.

```

#include <stdio.h>
#include <string.h>

/* ----- Full (simplified, textbook-style) DES
implementation ----- */

```

```

int IP[64] = {58,50,42,34,26,18,10,2, 60,52,44,36,28,20,12,4,
             62,54,46,38,30,22,14,6, 64,56,48,40,32,24,16,8,
             57,49,41,33,25,17,9,1, 59,51,43,35,27,19,11,3,
             61,53,45,37,29,21,13,5, 63,55,47,39,31,23,15,7};
int FP[64] = {40,8,48,16,56,24,64,32, 39,7,47,15,55,23,63,31,
             38,6,46,14,54,22,62,30, 37,5,45,13,53,21,61,29,
             36,4,44,12,52,20,60,28, 35,3,43,11,51,19,59,27,
             34,2,42,10,50,18,58,26, 33,1,41,9,49,17,57,25};
int E[48] = {32,1,2,3,4,5, 4,5,6,7,8,9, 8,9,10,11,12,13,
12,13,14,15,16,17,
             16,17,18,19,20,21, 20,21,22,23,24,25,
24,25,26,27,28,29, 28,29,30,31,32,1};
int P[32] = {16,7,20,21,29,12,28,17,1,15,23,26,5,18,31,10,
             2,8,24,14,32,27,3,9,19,13,30,6,22,11,4,25};
int PC1[56] = {57,49,41,33,25,17,9, 1,58,50,42,34,26,18,
10,2,59,51,43,35,27,
             19,11,3,60,52,44,36, 63,55,47,39,31,23,15,
7,62,54,46,38,30,22,
             14,6,61,53,45,37,29, 21,13,5,28,20,12,4};
int PC2[48] = {14,17,11,24,1,5, 3,28,15,6,21,10, 23,19,12,4,26,8,
16,7,27,20,13,2,
             41,52,31,37,47,55, 30,40,51,45,33,48,
44,49,39,56,34,53, 46,42,50,36,29,32};
int shifts[16] = {1,1,2,2,2,2,2,1,2,2,2,2,2,2,1};
int Sbox[8][4][16] = {
    {{14,4,13,1,2,15,11,8,3,10,6,12,5,9,0,7},
    {0,15,7,4,14,2,13,1,10,6,12,11,9,5,3,8},
    {4,1,14,8,13,6,2,11,15,12,9,7,3,10,5,0},
    {15,12,8,2,4,9,1,7,5,11,3,14,10,0,6,13}},
    {{15,1,8,14,6,11,3,4,9,7,2,13,12,0,5,10},
    {3,13,4,7,15,2,8,14,12,0,1,10,6,9,11,5},
    {0,14,7,11,10,4,13,1,5,8,12,6,9,3,2,15},
    {13,8,10,1,3,15,4,2,11,6,7,12,0,5,14,9}},
    {{10,0,9,14,6,3,15,5,1,13,12,7,11,4,2,8},
    {13,7,0,9,3,4,6,10,2,8,5,14,12,11,15,1},
    {13,6,4,9,8,15,3,0,11,1,2,12,5,10,14,7},
    {1,10,13,0,6,9,8,7,4,15,14,3,11,5,2,12}},
    {{7,13,14,3,0,6,9,10,1,2,8,5,11,12,4,15},
    {13,8,11,5,6,15,0,3,4,7,2,12,1,10,14,9},
    {10,6,9,0,12,11,7,13,15,1,3,14,5,2,8,4},
    {3,15,0,6,10,1,13,8,9,4,5,11,12,7,2,14}},
    {{2,12,4,1,7,10,11,6,8,5,3,15,13,0,14,9},
    {14,11,2,12,4,7,13,1,5,0,15,10,3,9,8,6},
    {4,2,1,11,10,13,7,8,15,9,12,5,6,3,0,14},
    {11,8,12,7,1,14,2,13,6,15,0,9,10,4,5,3}},
    {{12,1,10,15,9,2,6,8,0,13,3,4,14,7,5,11},
    {10,15,4,2,7,12,9,5,6,1,13,14,0,11,3,8},
    {9,14,15,5,2,8,12,3,7,0,4,10,1,13,11,6},
    {4,3,2,12,9,5,15,10,11,14,1,7,6,0,8,13}},
    {{4,11,2,14,15,0,8,13,3,12,9,7,5,10,6,1},
    {13,0,11,7,4,9,1,10,14,3,5,12,2,15,8,6},
    {1,4,11,13,12,3,7,14,10,15,6,8,0,5,9,2},
    {6,11,13,8,1,4,10,7,9,5,0,15,14,2,3,12}},
    {{13,2,8,4,6,15,11,1,10,9,3,14,5,0,12,7},
    {1,15,13,8,10,3,7,4,12,5,6,11,0,14,9,2},
    {7,11,4,1,9,12,14,2,0,6,10,13,15,3,5,8},
    {2,1,14,7,4,10,8,13,15,12,9,0,3,5,6,11}}};

void permute(int *in, int *out, int *table, int n) { for (int
i=0;i<n;i++) out[i]=in[table[i]-1]; }
void leftShift(int *bits, int n, int s) { int t[28]; for(int
i=0;i<n;i++) t[i]=bits[(i+s)%n]; for(int i=0;i<n;i++) bits[i]=t[i];
}

void generateRoundKeys(int key64[64], int roundKeys[16][48]) {
    int key56[56]; permute(key64, key56, PC1, 56);
    int C[28], D[28];
    memcpy(C, key56, 28*sizeof(int)); memcpy(D, key56+28,
28*sizeof(int));

```

```

        for (int r = 0; r < 16; r++) {
            leftShift(C, 28, shifts[r]); leftShift(D, 28, shifts[r]);
            int CD[56]; memcpy(CD, C, 28*sizeof(int)); memcpy(CD+28, D,
28*sizeof(int));
            permute(CD, roundKeys[r], PC2, 48);
        }
    }

    void feistel(int R[32], int K[48], int out[32]) {
        int er[48]; permute(R, er, E, 48);
        int xr[48]; for (int i=0;i<48;i++) xr[i]=er[i]^K[i];
        int sout[32];
        for (int b = 0; b < 8; b++) {
            int *blk = xr + b*6;
            int row = blk[0]*2 + blk[5];
            int col = blk[1]*8 + blk[2]*4 + blk[3]*2 + blk[4];
            int val = Sbox[b][row][col];
            for (int bit = 0; bit < 4; bit++) sout[b*4+bit] = (val >>
(3-bit)) & 1;
        }
        permute(sout, out, P, 32);
    }

    void desProcess(int input[64], int roundKeys[16][48], int
output[64], int decrypt) {
        int ip[64]; permute(input, ip, IP, 64);
        int L[32], R[32];
        memcpy(L, ip, 32*sizeof(int)); memcpy(R, ip+32, 32*sizeof(int));
        for (int round = 0; round < 16; round++) {
            int *K = decrypt ? roundKeys[15-round] : roundKeys[round];
            int fout[32]; feistel(R, K, fout);
            int newR[32]; for (int i=0;i<32;i++) newR[i]=L[i]^fout[i];
            memcpy(L, R, 32*sizeof(int)); memcpy(R, newR,
32*sizeof(int));
        }
        int preOutput[64]; memcpy(preOutput, R, 32*sizeof(int));
        memcpy(preOutput+32, L, 32*sizeof(int));
        permute(preOutput, output, FP, 64);
    }

    void bytesToBits(unsigned char *bytes, int nbytes, int *bits) {
        for (int i = 0; i < nbytes; i++)
            for (int b = 0; b < 8; b++) bits[i*8+b] = (bytes[i] >> (7-
b)) & 1;
    }

    void bitsToBytes(int *bits, int nbits, unsigned char *bytes) {
        for (int i = 0; i < nbits/8; i++) {
            bytes[i] = 0;
            for (int b = 0; b < 8; b++) bytes[i] = (bytes[i] << 1) |
bits[i*8+b];
        }
    }

    int main() {
        unsigned char keyBytes[8] =
{0x13,0x34,0x57,0x79,0x9B,0xBC,0xDF,0xF1};
        unsigned char plainBytes[8] =
{'D','E','S','B','L','O','C','K'};
        int keyBits[64], plainBits[64], cipherBits[64],
decryptedBits[64];
        unsigned char cipherBytes[8], decryptedBytes[8];

        bytesToBits(keyBytes, 8, keyBits);
        bytesToBits(plainBytes, 8, plainBits);

        int roundKeys[16][48];
        generateRoundKeys(keyBits, roundKeys);

        desProcess(plainBits, roundKeys, cipherBits, 0);
    }

```

```

bitsToBytes(cipherBits, 64, cipherBytes);
printf("Ciphertext (hex): ");
for (int i = 0; i < 8; i++) printf("%02X ", cipherBytes[i]);
printf("\n");

desProcess(cipherBits, roundKeys, decryptedBits, 1);
bitsToBytes(decryptedBits, 64, decryptedBytes);
decryptedBytes[8] = '\0';
printf("Decrypted (text): %.8s\n", decryptedBytes);
return 0;
}

```

This is a complete, self-contained implementation of the DES block cipher covering the initial permutation, 16-round Feistel structure with the standard 8 S-boxes, the P-box permutation, the PC-1/PC-2 key-schedule permutations with the correct per-round left-shift schedule, and the final permutation — using the **same function desProcess** for both encryption (decrypt=0) and decryption (decrypt=1, which simply applies the 16 round keys in reverse order), exactly matching the requirement that “the same algorithm and key are used for encryption and decryption, with minor differences.”

Questions 34, 35, 36, 37, 39, 40 — Repeated Questions

The paper repeats several earlier questions verbatim later in the list. Their C programs are identical to the ones already given above:

Repeated Question	Same as
Q34 (ECB/CBC/CFB padding motivation)	See Question 21
Q35 (One-time-pad Vigenère, “send more money”)	See Question 14
Q36 (Affine Caesar cipher, one-to-one requirement)	See Question 5
Q37 (Frequency attack on monoalphabetic substitution)	See Question 16
Q38 (Hill cipher known-plaintext attack)	See Question 13
Q39 (Frequency attack on additive/Caesar cipher)	See Question 15
Q40 (Frequency attack on monoalphabetic substitution)	See Question 16

End of list — all 40 questions from the University Practical Examination (Jan 2023) list of programs are covered above, either with a complete, compilable C program, or (for the purely analytical sub-parts) with a worked answer alongside a demonstrative C program.